

Ensamblador ARM instrucciones

Instrucción Compara
Estructuras de control de flujo
Ejemplos ARM

cmp

- Instrucción de comparación

Mnemónico	Instrucción	{S} Flags	Acción
CMP {cond} Rn, Operand2	Compare	N Z C V	CPSR flags := Rn - Op2

CMP es una instrucción de comparación que no almacena un resultado, pero actualiza los flags del procesador (Zero, Negative, Carry, Overflow)

Se usa antes de una instrucción condicional como BEQ, BNE, BGT, BLE, etc.

```
cmp r1, r2
ble rama_else
```

Control de flujo / instrucciones condicionales

Después de una comparación con CMP, se puede alterar el flujo del programa con saltos condicionales:

BEQ (Branch if Equal), BNE (Branch if Not Equal), BGT, BLT, BGE, BLE, etc.

cmp

Toma de decisiones / lógica de control

Permite implementar estructuras como:

if / else

while / for

switch / case

Manipulación de flags del procesador

La comparación modifica:

Z (Zero): se activa si el resultado es 0 ($R_n == R_m$)

N (Negative): se activa si el resultado es negativo

C (Carry) y V (Overflow) también se pueden afectar

cmp

- Instrucción aritmética **cmp** comparar

Mnemónico	Instrucción	{S} Flags	Acción
CMP {cond} Rn, Operand2	Compare	N Z C V	CPSR flags := Rn - Op2

Sintaxis

sub, m, Oper2

Donde:

- **m** es el registro que contiene el primer operando
- **Oper2** segundo operando.

```
cmp r1, #10 // compara el contenido de r1 con 10
```

```
cmp r3, r2
```

```
cmp r1, #0  
ble fin
```

Mapeo de estructuras de control de flujo

Los lenguajes de alto nivel manejan abstracciones que facilitan la programación, como ser: **Expresiones, Estructuras de control, Tipos de datos, Manejo de memoria**

Al programar en lenguaje ensamblador no contamos con la mayoría de estas facilidades, sin embargo podemos aplicar criterios similares para generar código ordenado y disminuir la posibilidad de introducir errores

Para esto vamos a ver cómo se pueden traducir ciertas **estructuras de control de flujo** a lenguaje ensamblador, de manera similar a la tarea que hacen los compiladores

Códigos de condición

Código	Sufijo	Flags	Significado
0000	eq	Z set	Igual
0001	ne	Z clear	No Igual
0011	cs	C set	Mayor o igual sin signo
0011	cc	C clear	Menor sin signo
0100	mi	N set	Negativo
0101	pl	N clear	Positivo o cero
0110	vs	V set	Overflow
0111	vc	V clear	No overflow
1000	hi	C set and Z clear	mayor sin signo
1001	ls	C clear and Z set	menor o igual sin signo
1010	ge	N equals V	Mayor o igual
1011	lt	N not equal to V	Menor que
1100	gt	Z clear AND (N equal to V)	Mayor que
1101	le	Z set OR (N not equal to V)	menor o igual que
1110	al	(ignored)	always, siempre

Sentencia if-then-else

```
...  
if(a>b) {  
    a = a-b;  
} else {  
    a = b-a;  
}  
...
```

```
ldr r1, =a  
ldr r1, [r1]  
ldr r2, =b  
ldr r2, [r2]  
cmp r1, r2  
ble rama_else // cond opuesta else  
sub r1, r1, r2 // calculo a-b  
ldr r3, =a  
str r1, [r3]  
b fin_if // no ejecuta else rama_else:  
rama_else:  
sub r1, r2, r1  
ldr r3, =a  
str r1, [r3]  
fin_if:
```

Ciclo while

```
...
```

```
while(a>0) {
```

```
    c++;
```

```
    a = a-1;
```

```
}
```

```
return 0;
```

```
...
```

```
ldr r4, =a
```

```
ldr r1, [r4]
```

```
ciclo:
```

```
cmp r1, #0
```

```
ble fin //usa el salto opuesto
```

```
ldr r2, =c
```

```
ldr r3, [r2]
```

```
add r3, #1
```

```
str r3, [r2]
```

```
sub r1, #1
```

```
str r1, [r4] // almacena el valor de a
```

```
b ciclo
```

```
fin:
```

```
mov r0, #0
```

```
bx lr
```


For

```
...  
  
for (i=0; i<8;i++)  
{  
    r3 = r1 + r2;  
}  
...
```

```
...  
  
mov r0, #0 //r0 actúa como índice i  
for:  
    cmp r0, #8  
    bge fin_for  
    add r3, r1, r2  
    add r0, #1  
    b for  
fin_for:  
...
```

Repeat until

```
...  
  
repeat until i=0  
{  
    r3 = r1 + r2;  
}  
...
```

```
...  
  
mov r0, #0 //r0 actúa como índice i  
repeat:  
    add r3, r1, r2  
    add r0, #1  
    cmp r0, #8  
    bne repeat  
  
...
```

Switch/Case

```
...  
  
switch(a) {  
case 1:  
    c = 10;  
    break;  
case 2:  
    c = 20;  
    break;  
default:  
    c = 0;  
}  
  
...
```

```
...  
ldr r1, =a  
ldr r1, [r1]  
ldr r2, =c    //r2 apunta a c  
cmp r1, #1  
beq caso1  
cmp r1, #2  
beq caso2  
b casodef  
caso1:  
    mov r3, #10  
    str r3, [r2]  
    b continuar  
caso2:  
    mov r3, #20  
    str r3, [r2]  
    b continuar  
casodef:  
    mov r3, #0  
    str r3, [r2]  
continuar:
```

Switch/Case (continua)

```
...  
  
switch(a) {  
  case 1:  
    c = 10;  
    break;  
  case 2:  
    c = 20;  
    break;  
  default:  
    c = 0;  
}  
  
...
```

```
...  
caso1:  
    mov r3, #10  
    str r3, [r2]  
    b continuar  
caso2:  
    mov r3, #20  
    str r3, [r2]  
    b continuar  
casodef:  
    mov r3, #0  
    str r3, [r2]  
continuar:  
  
...
```

Tabla 1.7. Condiciones asociadas a las instrucciones de salto

Mnemotécnico	Descripción	Flags
EQ	Igual	Z=1
NE	Distinto	Z=0
HI	Mayor que (sin signo)	C=1 & Z=0
LS	Menor o igual que (sin signo)	C=0 or Z=1
GE	Mayor o igual que (con signo)	N=V
LT	Menor que con signo	N!=V
GT	Mayor que con signo	Z=0 & N=V
LE	Menor o igual que (con signo)	Z=1 or N!=V
(vacío)	Siempre (incondicional)	

Tabla 1.8. Ejemplos de instrucciones de salto condicional

B etiqueta	Salta siempre a la dirección dada por la etiqueta
BEQ etiqueta	Salta a la etiqueta si el flag Z está activado
BLS etiqueta	Salta a la etiqueta si el flag Z o el N están activados
BHI etiqueta	Salta a etiqueta si C =1 y Z = 0

Tabla 1.9. Traducción de lenguaje de alto nivel a lenguaje ensamblador

<pre> if (R1 > R2) R3 = R4 + R5; else R3 = R4 - R5 sigue la ejecución normal </pre>	<pre> CMP R1, R2 BLE else ADD R3, R4, R5 B fin_if else: SUB R3, R4, R5 fin_if: <i>sigue la ejecución normal</i> </pre>
<pre> for (i=0; i<8; i++) { R3 = R1 + R2; } sigue la ejecución normal </pre>	<pre> MOV R0, #0 @R0 actúa como índice i for: CMP R0, #8 BGE fin_for ADD R3, R1, R2 ADD R0, #1 B for fin_for: <i>sigue la ejecución normal</i> </pre>
<pre> repeat until i=8 { R3 = R1 + R2; } sigue la ejecución normal </pre>	<pre> MOV R0, #0 @R0 actúa como índice i repeat: ADD R3, R1, R2 ADD R0, #1 CMP R0, #8 BNE repeat <i>sigue la ejecución normal</i> </pre>
<pre> while (R1 < R2) { R2= R2-R3; } sigue la ejecución normal </pre>	<pre> while: CMP R1, R2 BGE fin_w SUB R2, R2, R3 B while fin_w: <i>sigue la ejecución normal</i> </pre>